Visual Recognition using Deep Learning

Homework 2: Digit Detection Problem

Muhammad Rayhan Athaillah

Student ID: 313540001

April 2026

Abstract

GitHub Repository: https://github.com/RayhanHaqi/visual_recognition-hw2

1 Introduction

The Digit Detection Problem presents a unique challenge in the domain of visual recognition, requiring a model capable of accurately localizing and classifying dense numbers within varied backgrounds. Furthermore, the dataset provided for this task contains images with extreme widescreen dimensions, posing a significant challenge to standard object detection pipelines optimized for square aspect ratios.

To tackle this problem, this project utilizes the Real-Time DEtection TRansformer v2 (RT-DETRv2). Historically, real-time object detection has been dominated by CNN-based architectures like the YOLO family. While highly efficient, these traditional models rely heavily on anchor boxes and heuristic post-processing steps like Non-Maximum Suppression (NMS) to filter out redundant bounding boxes. NMS introduces latency and often struggles in highly dense environments where objects overlap.

The paradigm shifted significantly with the introduction of the DEtection TRansformer (DETR). DETR revolutionized the field by casting object detection as a direct set prediction problem. By utilizing a Transformer encoder-decoder architecture and a bipartite matching loss, DETR completely eliminated the need for NMS, offering a highly deterministic approach to object localization. However, early iterations of DETR suffered from exceedingly slow training convergence and

high computational overhead, especially when processing high-resolution images, making them unsuitable for real-time applications.

The evolution from DETR to RT-DETR (and subsequently RT-DETRv2) bridged this gap. RT-DETRv2 introduces an efficient hybrid encoder that effectively processes multi-scale features while reducing computational redundancy. It achieves the real-time inference speeds of YOLO while retaining the NMS-free, high-accuracy characteristics of the original DETR architecture. This makes RT-DETRv2 exceptionally suited for the dense Digit Detection Problem.

2 Methodology

2.1 Data Pre-Processing and Rectangular Training

An exploratory data analysis of the dataset revealed an average image resolution of approximately 60×140 pixels, yielding an aspect ratio of roughly $3:7$. Resizing these images to the default 640×640 resolution squashes the digits vertically by a factor of 10. To preserve the geometric integrity of the numbers, **Rectangular Training** was implemented. All images were resized to 320×640 during both training and validation. This maintains the natural proportions of the digits while fulfilling the neural network's requirement for dimensions to be multiples of 32.

2.2 Strategic Data Augmentation

To prevent altering the ground-truth bounding box coordinates, spatial/geometric augmentations (e.g., Random Horizontal Flip, Random Rotation) were strictly avoided. Instead, pixel-level augmentations were applied to increase model robustness:

- **ColorJitter:** Randomly alters brightness, contrast, and saturation to simulate diverse lighting conditions.
- **RandomErasing ($p = 0.3$):** Randomly drops out rectangular pixel patches, forcing the model to learn digit features even under partial occlusion.

2.3 Hyper-parameter Experiments

To systematically determine the optimal configuration for the RT-DETRv2 architecture (ResNet-50 backbone), a series of empirical experiments were conducted across several key hyper-parameters using multiple GPUs training.

- **Object Queries (q):** Unlike YOLO, DETR-based models predict a fixed set of queries. Given the dense nature of the digit clusters in the dataset, experiments were conducted comparing $q = 300$, $q = 1000$, and $q = 1500$ to ensure the model had sufficient capacity to detect all instances without incurring unnecessary computational overhead.

- **Learning Rate (LR) and Optimizer:** Using the AdamW optimizer with a OneCycleLR scheduler, base learning rates of 1×10^{-4} and 5×10^{-5} were tested to balance fast convergence and stable gradient descent.

- **Weight Decay (WD):** To prevent the model from memorizing the training data (overfitting), weight decay coefficients were rigorously evaluated at 5×10^{-4} and 1×10^{-3}.

3 Results and Discussion

3.1 Hyper-parameter Comparative Analysis

The training logs from the hyper-parameter experiments were extracted and plotted to rank the configurations based on Validation Loss convergence and Mean Average Precision (mAP@50-95).

Figure 1: Comparative analysis of training and validation curves across different hyper-parameter configurations (Queries, LR, and Weight Decay).

As illustrated in Figure 1, the configuration utilizing $q = 1000$, a learning rate of 1×10^{-4}, and a weight decay of 1×10^{-3} demonstrated the most stable convergence. Increasing the queries to 1500 did not yield a proportional increase in mAP, indicating that 1000 queries are sufficient for the maximum digit density present in the dataset.

3.2 Baseline Performance and the Overfitting Gap

Initial training runs utilizing the default 640×640 resolution achieved a strong local validation mAP of 0.47. However, performance on the CodaBench test set consistently stalled at 0.40. This 0.06 gap strongly indicated an overfitting syndrome where the model behaved too aggressively, predicting numerous false positives on empty background regions.

3.3 Innovative Strategy 1: Strict Background Penalty (`eos_coef`)

To address the false positive issue, the End-of-Sequence coefficient (`eos_coef` or No-Object Weight) in the bipartite matching loss was radically tuned. By increasing the `eos_coef` from its default value of 0.0001 to 0.005, the model was subjected to a much stricter penalty for predicting objects in empty spaces. This forced the model to be highly confident before outputting a bounding box, acting as an effective filter against background noise and directly improving the Precision metric on the CodaBench evaluation.

3.4 Innovative Strategy 2: Efficiency via Aspect Ratio Preservation

The transition to Rectangular Training (320×640) yielded a twofold advantage. First, it eliminated the geometric distortion, allowing the Transformer's attention mechanisms to learn the true shapes of the digits. Second, it reduced the computational load per epoch by 50%, allowing for faster experimentation and the utilization of larger batch sizes ($BS = 32$) without triggering Out-Of-Memory (OOM) errors.

4 Conclusion